

Основы ASP NET Core

Продолжаем изучать

Метод Use

Метод расширения **Use()** добавляет компонент middleware, который позволяет передать обработку запроса далее следующим в конвейере компонентам. Он имеет следующие версии

```
public static IApplicationBuilder Use(this IApplicationBuilder app, Func<HttpContext, Func<Task>, Task> middleware);  
public static IApplicationBuilder Use(this IApplicationBuilder app, Func<HttpContext, RequestDelegate, Task> middleware)
```

Метод **Use()** реализован как метод расширения для типа **IApplicationBuilder**, соответственно мы можем вызвать данный метод у объекта **WebApplication** для добавления middleware в приложение. В обеих версиях метод Use принимает некоторое действие, которое имеет два параметра и возвращает объект Task.

Первый параметр делегата Func, который передается в метод **Use()**, представляет объект **HttpContext**. Этот объект позволяет получить данные запроса и управлять ответом клиенту.

Второй параметр делегата представляет другой делегат - **Func<Task>** или **RequestDelegate**. Этот делегат представляет следующий в конвейере компонент middleware, которому будет передаваться обработка запроса.

В общем случае применение метода **Use()** выглядит следующим образом:

```
app.Use(async (context, next) =>  
{  
    // действия перед передачи запроса в следующий middleware  
    await next.Invoke();  
    // действия после обработки запроса следующим middleware  
});
```

Метод Use

Работа middleware разбивается на две части:

- Middleware выполняет некоторую начальную обработку запроса до вызова `await next.Invoke()`
- Затем вызывается метод `next.Invoke()`, который передает обработку запроса следующему компоненту в конвейере
- Когда следующий в конвейере компонент закончил обработку запроса возвращается в обратно в текущий компонент, и выполняются действия, которые идут после вызова `await next.Invoke()`

Таким образом, middleware в методе `Use` выполняет действия до следующего в конвейере компонента и после него.

Рассмотрим метод `Use()` на примере:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

string date = "";

app.Use(async(context, next) =>
{
    date = DateTime.Now.ToShortDateString();
    await next.Invoke();           // вызываем middleware из app.Run
    Console.WriteLine($"Current date: {date}"); // Current date: 23.03.2026
});

app.Run(async(context) => await context.Response.WriteAsync($"Date: {date}"));

app.Run();
```

Метод Use

В данном случае мы используем перегрузку метода `Use`, которая в качестве параметров принимает контекст запроса - объект `HttpContext` и делегат `Func<Task>`, который представляет собой ссылку на следующий в конвейере компонент `middleware`.

`Middleware` в методе `app.Use()` реализует простейшую задачу - присваивает переменной `date` текущую дату в виде строки и затем передает обработку запроса следующим компонентам `middleware` в конвейере. То есть при вызове `await next.Invoke()` обработка запроса перейдет к тому компоненту, который установлен в методе `app.Run()`. В итоге обработка запроса будет выглядеть следующим образом:

Вызов компонента `app.Use`

Установка значения переменной `date`:

```
date = DateTime.Now.ToShortDateString();
```

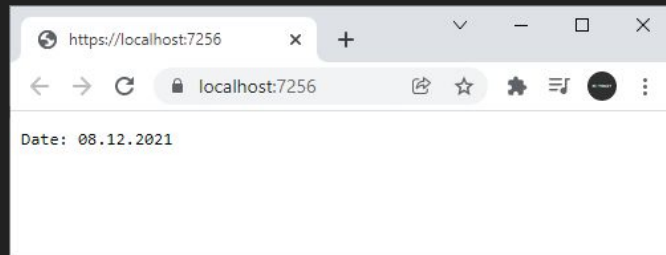
Вызов `await next.Invoke()`. Управление переходит следующему компоненту в конвейере - к `app.Run`.

В `middleware` из `app.Run()` отправляет клиенту текущую дату в качестве ответа с помощью метода `context.Response.WriteAsync()`:

```
await context.Response.WriteAsync($"Date: {date}");
```

Метод `app.Run` закончил свою работу, и управление обработкой возвращается к `middleware` в методе `app.Use`. Начинает выполняться та часть кода, которая идет после `await next.Invoke()`. В этой части выполняется условное логирование - на консоль выводится значение переменной `date`:

```
Console.WriteLine($"Current date: {date}");
```



Метод Use. Отправка ответа

При использовании метода Use и передаче выполнения следующему делегату следует учитывать, что не рекомендуется вызывать метод `next.Invoke` после метода `Response.WriteAsync()`. Компонент middleware должен либо генерировать ответ с помощью `Response.WriteAsync`, либо вызывать следующий делегат посредством `next.Invoke`, но не выполнять оба этих действия одновременно. Так как согласно документации последующие изменения объекта `Response` могут привести к нарушению протокола, например, будет послано больше байт, чем указано в заголовке `Content-Length`, либо могут привести к нарушению тела ответа, например, футер страницы HTML запишется в CSS-файл.

То есть к примеру следующая обработка запроса не рекомендуется:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("<p>Hello world!</p>");
    await next.Invoke();
});

app.Run(async (context) =>
{
    //await Task.Delay(10000); // можно поставить задержку
    await context.Response.WriteAsync("<p>Good bye, World...</p>");
});

app.Run();
```

Использование делегат RequestDelegate

В примере выше использовалась версия метода **Use()**, которая использует делегат **Func<Task>**. Подобным образом можно использовать и другую версию, где используется делегат **RequestDelegate**. Единственное - при вызове делегата (то есть фактически следующего в конвейере компонента) необходимо передавать делегату объект **HttpContext**:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

string date = "";

app.Use(async(context, next) =>
{
    date = DateTime.Now.ToShortDateString();
    await next.Invoke(context);           // здесь next - RequestDelegate
    Console.WriteLine($"Current date: {date}"); // Current date: 23.03.2026
});

app.Run(async(context) => await context.Response.WriteAsync($"Date: {date}"));

app.Run();
```

Терминальный компонент middleware

Middleware в методе **Use()** необязательно должен вызывать к следующему в конвейере компоненту. Вместо этого он может завершить обработку запроса. В этом случае он может выступать в роли такого же терминального компонента middleware, а и компоненты из метода **Run()**. Например:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Use(async(context, next) =>
{
    string? path = context.Request.Path.Value?.ToLower();
    if (path == "/date")
    {
        await context.Response.WriteAsync($"Date: {DateTime.Now.ToShortDateString()}");
    }
    else
    {
        await next.Invoke();
    }
});

app.Run(async(context) => await context.Response.WriteAsync($"Hello World"));
app.Run();
```

Здесь middleware в app.Use проверяет запрошенный адрес - если он содержит "/date", то клиенту отправляется текущая дата. Иначе обработка запроса передается дальше в app.Run.

Причем в принципе мы можем использовать компонент в app.Use как единственный и соответственно терминальный:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();
app.Use(async (HttpContext context, Func<Task> next) =>
{
    await context.Response.WriteAsync("Hello Work!");
});

app.Run();
```

Однако в данном случае для большей производительности лучше использовать **app.Run()**, если нам надо определить лишь один компонент, который в принципе не передает запрос дальше по конвейеру.

Вынесение компонентов в методы

Также можно вынести все inline-компоненты в отдельные методы:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Use(GetDate);
app.Run(async (context) => await context.Response.WriteAsync("Hello World"));
app.Run();

async Task GetDate(HttpContext context, Func<Task> next)
{
    string? path = context.Request.Path.Value?.ToLower();
    if (path == "/date")
    {
        await context.Response.WriteAsync($"Date: {DateTime.Now.ToShortDateString()}");
    }
    else
    {
        await next.Invoke();
    }
}
```

Подобным образом можно использовать и другую версию метода Use, в которой используется делегат RequestDelegate:

```
async Task GetDate(HttpContext context, RequestDelegate next)
{
    string? path = context.Request.Path.Value?.ToLower();
    if (path == "/date")
    {
        await context.Response.WriteAsync($"Date: {DateTime.Now.ToShortDateString()}");
    }
    else
    {
        await next.Invoke(context);
    }
}
```

Создание ветки конвейера. UseWhen и MapWhen

UseWhen

Метод **UseWhen()** на основании некоторого условия позволяет создать ответвление конвейера при обработке запроса:

```
public static IApplicationBuilder UseWhen (this
IApplicationBuilder app, Func<HttpContext, bool> predicate,
Action<IApplicationBuilder> configuration);
```

Как и Use(), метод **UseWhen()** реализован как метод расширения для типа IApplicationBuilder.

В качестве параметра он принимает делегат Func<HttpContext, bool> - некоторое условие, которому должен соответствовать запрос. В этот делегат передается объект HttpContext. А возвращаемым типом должен быть тип bool - если запрос соответствует условию, то возвращается true, иначе возвращается false.

Последний параметр метода - делегат Action<IApplicationBuilder> представляет некоторые действия над объектом IApplicationBuilder, который передается в делегат в качестве параметра.

Рассмотрим небольшой пример:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseWhen(
    context => context.Request.Path == "/time", // если путь
    запроса "/time"
    appBuilder =>
    {
        // логируем данные - выводим на консоль приложения
        appBuilder.Use(async (context, next) =>
        {
            var time = DateTime.Now.ToShortTimeString();
            Console.WriteLine($"Time: {time}");
            await next(); // вызываем следующий middleware
        });

        // отправляем ответ
        appBuilder.Run(async context =>
        {
            var time = DateTime.Now.ToShortTimeString();
            await context.Response.WriteAsync($"Time: {time}");
        });
    });

app.Run(async context =>
{
    await context.Response.WriteAsync("Hello World");
});

app.Run();
```

Создание ветки конвейера. UseWhen и MapWhen

Стоит отметить, что создание ветки происходит один раз при запуске приложения. Например, в примере выше мы видим, что получение времени производится в обоих middleware во встраиваемой ветки. Но что будет, если вынести получение времени во вне и не дублировать в каждом middleware, например, следующим образом

В этом случае время будет устанавливаться один раз - при запуске приложения и создании ветки в конвейер. Соответственно вне зависимости от того, сколько раз мы будем обращаться к приложению по пути "/time", мы будем получать одно и то же время.

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseWhen(
    context => context.Request.Path == "/time", // если путь
    appBuilder =>
    {
        var time = DateTime.Now.ToShortTimeString();
        // логируем данные - выводим на консоль приложения
        appBuilder.Use(async (context, next) =>
        {
            Console.WriteLine($"Time: {time}");
            await next(); // вызываем следующий middleware
        });

        // отправляем ответ
        appBuilder.Run(async context =>
        {
            await context.Response.WriteAsync($"Time: {time}");
        });
    });

app.Run(async context =>
{
    await context.Response.WriteAsync("Hello World");
});

app.Run();
```

Создание ветки конвейера. UseWhen и MapWhen

В примере выше ветка конвейера завершалась терминальным компонентом, поэтому остальные действия в основной части конвейера не выполнялись. Однако мы можем также передать запрос на обработку из ветки в основной поток конвейера

В данном случае, если запрос идет по пути "/time", сначала срабатывает ветка конвейера с компонентом, который логирует время на консоль. А затем выполняется компоненте из `app.Run()`, который отправляет сообщение "Hello World":

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseWhen(
    context => context.Request.Path == "/time", //
    условие: если путь запроса "/time"
    appBuilder =>
    {
        appBuilder.Use(async (context, next) =>
        {
            var time = DateTime.Now.ToShortTimeString();
            Console.WriteLine($"Time: {time}");
            await next(); // вызываем следующий
middleware
        });
    });

app.Run(async context =>
{
    await context.Response.WriteAsync("Hello World");
});

app.Run();
```

Создание ветки конвейера. UseWhen и MapWhen

MapWhen

Метод `MapWhen()`, как и метод `UseWhen()`, на основании некоторого условия позволяет создать ответвление конвейера:

```
public static IApplicationBuilder MapWhen (this
IApplicationBuilder app, Func<HttpContext, bool> predicate,
Action<IApplicationBuilder> configuration);
```

Метод `MapWhen()` также реализован как метод расширения для типа `IApplicationBuilder`, принимает те же параметры, что и `UseWhen()`, и работает во многом аналогичным образом:

Здесь опять же, если запрошен путь `"/time"`, то срабатывает ветка конвейера, созданная методом `app.MapWhen()`, в которой клиенту отправляется текущее время. Если путь запроса другой, то срабатывается основной поток конвейера, в котором отправляется сообщение `"Hello World"`.

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapWhen(
    context => context.Request.Path == "/time", //
    условие: если путь запроса "/time"
    appBuilder => appBuilder.Run(async context =>
    {
        var time =
DateTime.Now.ToShortTimeString();
        await
context.Response.WriteAsync($"current time:
{time}"));
    });

app.Run(async context =>
{
    await context.Response.WriteAsync("Hello
METANIT.COM");
});

app.Run();
```

Метод Map

Метод **Map()** применяется для создания ветки конвейера, которая будет обрабатывать запрос по определенному пути. Этот метод реализован как метод расширения для типа `IApplicationBuilder` и имеет ряд перегруженных версий. Например:

```
public static IApplicationBuilder Map (this
IApplicationBuilder app, string pathMatch,
Action<IApplicationBuilder> configuration);
```

В качестве параметра `pathMatch` метод принимает путь запроса, с которым будет сопоставляться ветка. А параметр `configuration` представляет делегат, в который передается объект `IApplicationBuilder` и в котором будет создаваться ветка конвейера.

Рассмотрим простой пример:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/time", appBuilder =>
{
    var time = DateTime.Now.ToShortTimeString();

    // логируем данные - выводим на консоль
    приложения
    appBuilder.Use(async(context, next) =>
    {
        Console.WriteLine($"Time: {time}");
        await next();    // вызываем следующий
        middleware
    });

    appBuilder.Run(async context => await
context.Response.WriteAsync($"Time: {time}"));
});

app.Run(async (context) => await
context.Response.WriteAsync("Hello World"));

app.Run();
```

Метод Map

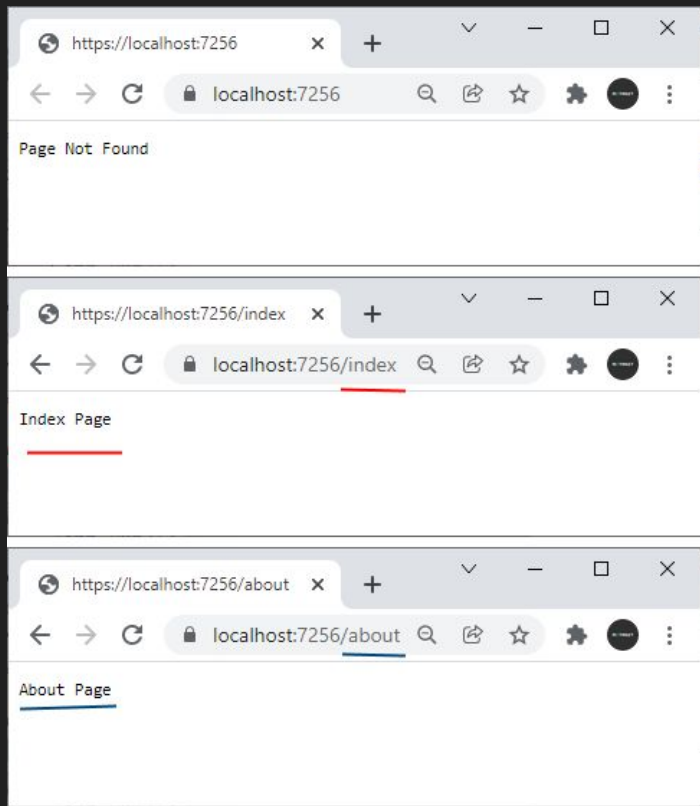
Подобным образом можно создавать ветки для разных путей:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/index", appBuilder =>
{
    appBuilder.Run(async context => await
context.Response.WriteAsync("Index Page"));
});
app.Map("/about", appBuilder =>
{
    appBuilder.Run(async context => await
context.Response.WriteAsync("About Page"));
});

app.Run(async (context) => await
context.Response.WriteAsync("Page Not Found"));

app.Run();
```



Метод Map

При необходимости создание веток конвейера можно вынести в отдельные методы:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/index", Index);
app.Map("/about", About);

app.Run(async(context) => await context.Response.WriteAsync("Page Not Found"));

app.Run();

void Index(IApplicationBuilder appBuilder)
{
    appBuilder.Run(async context => await context.Response.WriteAsync("Index"));
}

void About(IApplicationBuilder appBuilder)
{
    appBuilder.Run(async context => await context.Response.WriteAsync("About"));
}
```

Вложенные методы Map

Ветка конвейера, которая создается в методе **Map()**, может иметь вложенные ветки, которые обрабатывают подзапросы.

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

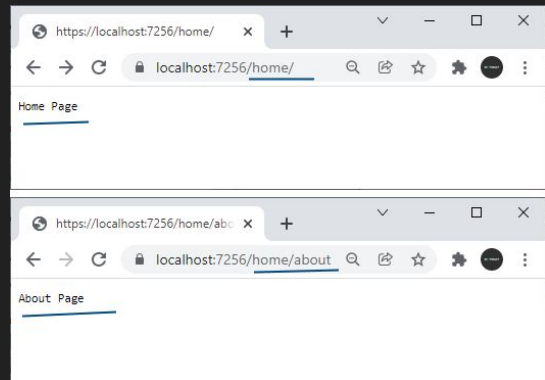
app.Map("/home", appBuilder =>
{
    appBuilder.Map("/index", Index); // middleware для
    "/home/index"
    appBuilder.Map("/about", About); // middleware для
    "/home/about"
    // middleware для "/home"
    appBuilder.Run(async (context) => await
context.Response.WriteAsync("Home Page"));
});

app.Run(async(context) => await
context.Response.WriteAsync("Page Not Found"));

app.Run();

void Index(IApplicationBuilder appBuilder)
{
    appBuilder.Run(async context => await
context.Response.WriteAsync("Index Page"));
}

void About(IApplicationBuilder appBuilder)
{
    appBuilder.Run(async context => await
context.Response.WriteAsync("About Page"));
}
```



Классы middleware

В прошлых темах используемые компоненты middleware фактически представляли методы, то есть так называемые **inline middleware**. Однако также ASP.NET Core позволяет определять middleware в виде отдельных классов.

Итак, добавим в проект новый класс, который назовем **TokenMiddleware** и который будет иметь следующий код

Класс middleware должен иметь конструктор, который принимает параметр типа **RequestDelegate**. Через этот параметр можно получить ссылку на тот делегат запроса, который стоит следующим в конвейере обработки запроса.

Также в классе должен быть определен метод, который должен называться либо **Invoke**, либо **InvokeAsync**. Причем этот метод должен возвращать объект **Task** и принимать в качестве параметра контекст запроса - объект **HttpContext**. Данный метод собственно и будет обрабатывать запрос.

Суть действия класса заключается в том, что мы получаем из запроса параметр "token". Если полученный токен равен строке "12345678", то передаем запрос дальше следующему компоненту, вызвав метод `_next.Invoke()`. Иначе возвращаем пользователю сообщение об ошибке.

```
public class TokenMiddleware
{
    private readonly RequestDelegate next;

    public TokenMiddleware(RequestDelegate next)
    {
        this.next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var token =
context.Request.Query["token"];
        if (token!="12345678")
        {
            context.Response.StatusCode = 403;
            await
context.Response.WriteAsync("Token is invalid");
        }
        else
        {
            await next.Invoke(context);
        }
    }
}
```

Классы middleware

Для добавления компонента middleware, который представляет класс, в конвейер обработки запроса применяется метод **UseMiddleware()**. Так, изменим файл **Program.cs** следующим образом:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseMiddleware<TokenMiddleware>();

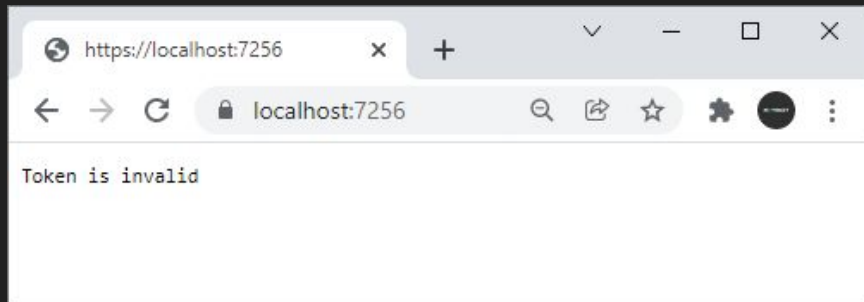
app.Run(async(context) => await context.Response.WriteAsync("Hello
World"));

app.Run();
```

С помощью метода **UseMiddleware<T>** в конструктор объекта **TokenMiddleware** будет внедряться объект для параметра **RequestDelegate next**. Поэтому явным образом передавать значение для этого параметра нам не нужно.

Запустим проект. И если мы не передадим через строку запроса параметр **token** или передадим для него значение, отличное от "12345678", то браузер отобразит ошибку

Если же будет передан корректный токен, то вызов **app.UseMiddleware<TokenMiddleware>()** передаст обработку запроса в компонент middleware из **app.Run()**



Метод расширения для встраивания middleware

Также нередко для встраивания подобных компонентов middleware определяются специальные методы расширения. Так, добавим в проект новый класс, который назовем TokenExtensions:

```
public static class TokenExtensions
{
    public static IApplicationBuilder UseToken(this IApplicationBuilder
builder)
    {
        return builder.UseMiddleware<TokenMiddleware>();
    }
}
```

Здесь создается метод расширения для типа IApplicationBuilder. И этот метод встраивает компонент TokenMiddleware в конвейер обработки запроса. Как правило, подобные методы возвращают объект IApplicationBuilder.

Теперь применим этот метод в коде программы в Program.cs:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseToken();

app.Run(async(context) => await context.Response.WriteAsync("Hello
World"));

app.Run();
```

Передача параметра

Изменим класс `TokenMiddleware`, чтобы он извне получал образец токена для сравнения

Образец токена, с которым идет сравнения, устанавливается через конструктор. Чтобы передать его в конструктор, изменим класс `TokenExtensions`:

```
public static class TokenExtensions
{
    public static IApplicationBuilder UseToken(this IApplicationBuilder builder, string
pattern)
    {
        return builder.UseMiddleware<TokenMiddleware>(pattern);
    }
}
```

В метод `builder.UseMiddleware` можно передать набор значений, которые передаются в конструктор компонента `middleware`.

И при вызове метода расширения `UseToken` в него можно передать конкретное значение:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseToken("555555");

app.Run(async(context) => await context.Response.WriteAsync("Hello World"));

app.Run();
```

```
public class TokenMiddleware
{
    private readonly RequestDelegate next;
    string pattern;
    public TokenMiddleware(RequestDelegate next,
string pattern)
    {
        this.next = next;
        this.pattern = pattern;
    }

    public async Task InvokeAsync(HttpContext
context)
    {
        var token =
context.Request.Query["token"];
        if (token != pattern)
        {
            context.Response.StatusCode = 403;
            await
context.Response.WriteAsync("Token is invalid");
        }
        else
        {
            await next.Invoke(context);
        }
    }
}
```

Построение конвейера обработки запроса

Обычно для обработки запроса применяется не один, а несколько компонентов middleware. И в этом случае большую роль может играть порядок их помещения в конвейер обработки запроса, а также то, как они взаимодействуют с другими компонентами.

Кроме того, каждый компонент middleware может обрабатывать запрос до и после последующих в конвейере компонентов. Данное обстоятельство позволяет предыдущим компонентам корректировать результат обработки последующих компонентов.

Рассмотрим простейший пример. Определим следующий класс

RoutingMiddleware для обработки запроса

Этот компонент в зависимости от строки запроса возвращает либо определенную строку, либо устанавливает код ошибки.

```
public class RoutingMiddleware
{
    public RoutingMiddleware(RequestDelegate _)
    {
    }

    public async Task InvokeAsync(HttpContext context)
    {
        string path = context.Request.Path;
        if (path == "/index")
        {
            await
context.Response.WriteAsync("Home Page");
        }
        else if (path == "/about")
        {
            await
context.Response.WriteAsync("About Page");
        }
        else
        {
            context.Response.StatusCode = 404;
        }
    }
}
```

Построение конвейера обработки запроса

Допустим, мы хотим, чтобы пользователь был аутентифицирован при обращении к нашему приложению. Для этого добавим новый класс

AuthenticationMiddleware

Условно будем считать, что если в строке запроса есть параметр `token` и он имеет какое-нибудь значение, то пользователь аутентифицирован. А если он не аутентифицирован, то надо необходимо ограничить доступ пользователям к приложению. Если пользователь не аутентифицирован, то устанавливаем статусный код 403, иначе передаем выполнение запроса следующему в конвейере делегату.

```
public class AuthenticationMiddleware
{
    readonly RequestDelegate next;
    public AuthenticationMiddleware(RequestDelegate next)
    {
        this.next = next;
    }
    public async Task InvokeAsync(HttpContext context)
    {
        var token = context.Request.Query["token"];
        if (string.IsNullOrEmpty(token))
        {
            context.Response.StatusCode = 403;
        }
        else
        {
            await next.Invoke(context);
        }
    }
}
```

Построение конвейера обработки запроса

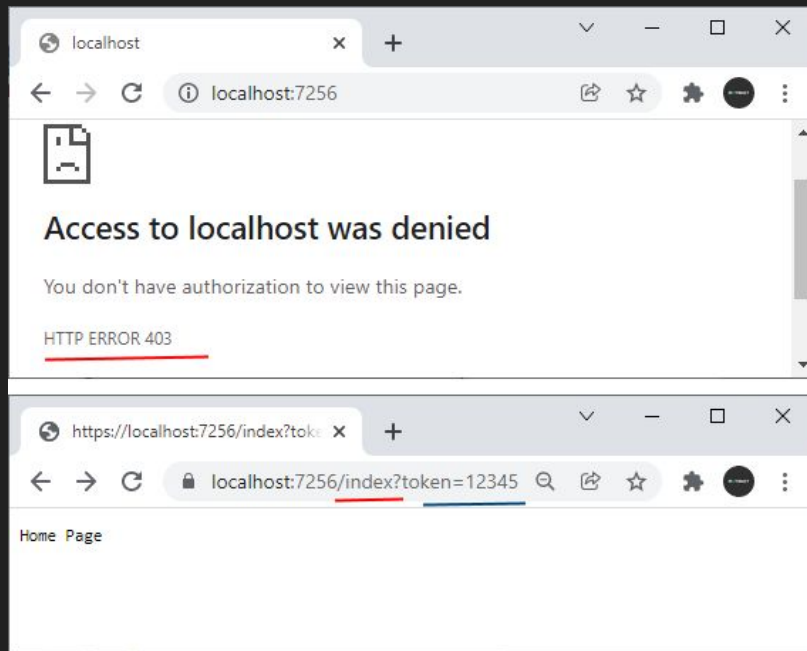
Поскольку компоненту `RoutingMiddleware` нет смысла обрабатывать запрос, если пользователь не аутентифицирован, то в конвейере компонент `AuthenticationMiddleware` должен быть помещен перед компонентом `RoutingMiddleware`:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseMiddleware<AuthenticationMiddleware>();
app.UseMiddleware<RoutingMiddleware>();

app.Run();
```

Таким образом, если мы сейчас запустим проект и обратимся по пути `"/index"` или `"/about"` и не передадим параметр `token`, то мы получим ошибку. Если же обратимся по пути `/index` или `/about` и передадим значение параметра `token`, то увидим искомый текст



Построение конвейера обработки запроса

Добавим еще один компонент middleware, который назовем **ErrorHandlingMiddleware**

В отличие от предыдущих двух компонентов `ErrorHandlingMiddleware` сначала передает запрос на выполнение последующим делегатам, а потом уже сам обрабатывает. Это возможно, поскольку каждый компонент обрабатывает запрос два раза: вначале вызывается та часть кода, которая идет до `await next.Invoke(context)`; а после завершения обработки последующих компонентов вызывается та часть кода, которая идет после `await next.Invoke(context)`; . И в данном случае для `ErrorHandlingMiddleware` важен результат обработки запроса последующими компонентами. В частности, он устанавливает сообщения об ошибках в зависимости от того, как статусный код установили другие компоненты. Поэтому `ErrorHandlingMiddleware` должен быть помещен первым из всех трех компонентов

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseMiddleware<ErrorHandlingMiddleware>();
app.UseMiddleware<AuthenticationMiddleware>();
app.UseMiddleware<RoutingMiddleware>();

app.Run();
```

```
public class ErrorHandlingMiddleware
{
    readonly RequestDelegate next;

    public ErrorHandlingMiddleware(RequestDelegate next)
    {
        this.next = next;
    }

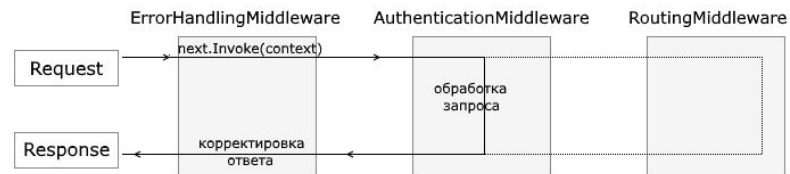
    public async Task InvokeAsync(HttpContext context)
    {
        await next.Invoke(context);
        if (context.Response.StatusCode == 403)
        {
            await context.Response.WriteAsync("Access Denied");
        }
        else if (context.Response.StatusCode == 404)
        {
            await context.Response.WriteAsync("Not Found");
        }
    }
}
```

Построение конвейера обработки запроса

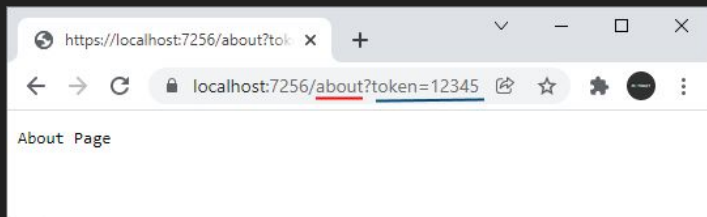
Схематично конвейер обработки запроса будет выглядеть следующим образом:



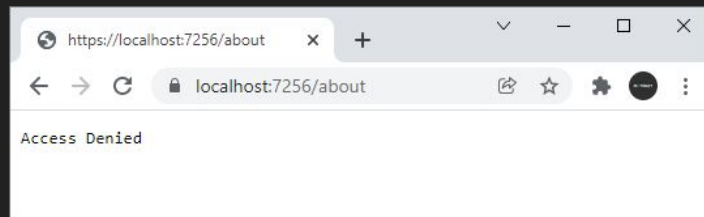
В то же время, если к приложению обратится пользователь, не указав в строке запроса параметр token, то **AuthenticationMiddleware** не будет передавать дальше запрос на обработку, а конвейер обработки будет выглядеть так:



В первом случае, если указан параметр token, то запрос будет обработан **RoutingMiddleware**:



Иначе пользователь получит ошибку 403:



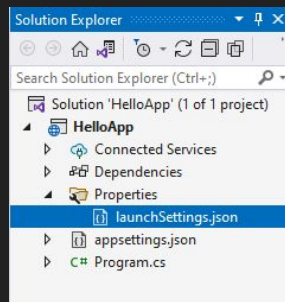
IWebHostEnvironment и окружение

Для взаимодействия с окружением, в котором запущено приложение, фреймворк ASP.NET Core предоставляет интерфейс **IWebHostEnvironment**. Этот интерфейс предлагает ряд свойств, с помощью которых мы можем получить информацию об окружении:

- **ApplicationName**: хранит имя приложения
- **EnvironmentName**: хранит название среды, в которой хостится приложение
- **ContentRootPath**: хранит путь к корневой папке приложения
- **WebRootPath**: хранит путь к папке, в которой хранится статический контент приложения. По умолчанию это папка `wwwroot`
- **ContentRootFileProvider**: возвращает реализацию интерфейса `Microsoft.AspNetCore.FileProviders.IFileProvider`, которая может использоваться для чтения файлов из папки `ContentRootPath`
- **WebRootFileProvider**: возвращает реализацию интерфейса `Microsoft.AspNetCore.FileProviders.IFileProvider`, которая может использоваться для чтения файлов из папки `WebRootPath`

При разработке мы можем использовать эти свойства. Но наиболее часто при разработке придется сталкиваться со свойством **EnvironmentName**. По умолчанию имеются три варианта значений для этого свойства: `Development`, `Staging` и `Production`. В проекте это свойство задается через установку переменной среды `ASPNETCORE_ENVIRONMENT`. Текущее значение данного параметра задается в файле **launchSettings.json**, который располагается в проекте в папке **Properties**.

Откроем данный файл:



```
{
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:56234",
      "sslPort": 44384
    }
  },
  "profiles": {
    "HelloApp": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl":
"https://localhost:7256;http://localhost:5256",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

IWebHostEnvironment и окружение

Здесь можно увидеть, что переменная "ASPNETCORE_ENVIRONMENT" встречается два раза - для запуска через IISExpress и для запуска через Kestrel. В обоих случаях она имеет значение Development. Но мы можем поменять значение этой переменной.

Для определения значения этой переменной для интерфейса IWebHostEnvironment определены специальные методы расширения:

- **IsEnvironment(string envName)**: возвращает true, если имя среды равно значению параметра envName
- **IsDevelopment()**: возвращает true, если имя среды - Development
- **IsStaging()**: возвращает true, если имя среды - Staging
- **IsProduction()**: возвращает true, если имя среды - Production

Данная функциональность позволяет нам выполнять определенный код в зависимости от того, на какой стадии находится приложение. Если приложение в процессе разработки, то мы можем выполнять один код, а при разветвлении для полноценного использования другой код

```
var builder = WebApplication.CreateBuilder();
WebApplication app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.Run(async (context) => await
context.Response.WriteAsync("In Development
Stage"));
}
else
{
    app.Run(async (context) => await
context.Response.WriteAsync("In Production
Stage"));
}
Console.WriteLine($"{app.Environment.EnvironmentName}");

app.Run();
```

IWebHostEnvironment и окружение

<https://github.com/dotnet/aspnetcore/blob/main/src/DefaultBuilder/src/WebApplicationBuilder.cs>

Так, если мы посмотрим на код класса `WebApplicationBuilder`, который применяется для создания приложения, то мы там можем увидеть там следующие строки:

```
if (context.HostingEnvironment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
```

Здесь если имя среды имеет значение "Development" (то есть приложение находится в состоянии разработки), то при ошибке разработчик увидит детальное описание ошибки. Если же приложение развернуто на хостинге и соответственно имеет другое имя хостирующей среды, то простой пользователь при ошибке ничего не увидит. Таким образом, в зависимости от стадии, на которой находится проект, мы можем скрывать или задействовать часть функционала приложения.

Если мы хотим поменять значение среды, необязательно изменять файл **launchSettings.json**. Это можно сделать также программно:

```
var builder = WebApplication.CreateBuilder();
WebApplication app = builder.Build();

app.Environment.EnvironmentName = "Production";
```

Определение своих состояний среды

Хотя по умолчанию среда может принимать три состояния: Development, Staging, Production, но мы можем при желании вводить новые значения. Например, нам надо отслеживать какие-то дополнительные состояния. Это можно сделать через изменение файла **launchSettings.json** либо программно.

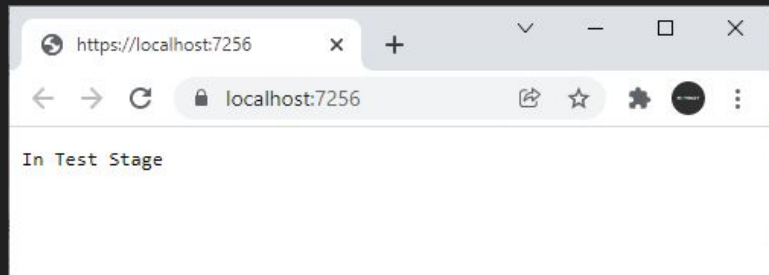
Например, изменим название среды на "Test" (значение может быть произвольное):

```
var builder = WebApplication.CreateBuilder();
WebApplication app = builder.Build();

app.Environment.EnvironmentName = "Test"; // изменяем название
среды на Test

if (app.Environment.IsEnvironment("Test")) // Если проект в состоянии
"Test"
{
    app.Run(async (context) => await context.Response.WriteAsync("In
Test Stage"));
}
else
{
    app.Run(async (context) => await context.Response.WriteAsync("In
Development or Production Stage"));
}

app.Run();
```



Определение своих состояний среды

Также можно изменить значение "ASPNETCORE_ENVIRONMENT" на "Test" или любое другое в файле **launchSettings.json** для используемого профиля

```
{
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:56234",
      "sslPort": 44384
    }
  },
  "profiles": {
    "HelloApp": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl":
"https://localhost:7256;http://localhost:5296",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Test"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Dependency Injection. Внедрение зависимостей и IServiceCollection

Dependency injection (DI) или внедрение зависимостей представляет механизм, который позволяет сделать взаимодействующие в приложении объекты слабосвязанными. Такие объекты связаны между собой через абстракции, например, через интерфейсы, что делает всю систему более гибкой, более адаптируемой и расширяемой.

В центре подобного механизма находится понятие **зависимость** - некоторая сущность, от которой зависит другая сущность. Например:

```
class Logger
{
    public void Log(string message) => Console.WriteLine(message);
}
class Message
{
    Logger logger = new Logger();
    public string Text { get; set; } = "";
    public void Print() => logger.Log(Text);
}
```

Dependency Injection. Внедрение зависимостей и IServiceCollection

Чтобы отвязать объект `Logger` от класса `Message`, мы можем создать абстракцию, которая будет представлять логгер, и передавать ее извне в объект `Message`:

```
interface ILogger
{
    void Log(string message);
}

class Logger : ILogger
{
    public void Log(string message) => Console.WriteLine(message);
}

class Message
{
    ILogger logger;
    public string Text { get; set; } = "";
    public Message(ILogger logger)
    {
        this.logger = logger;
    }
    public void Print() => logger.Log(Text);
}
```

Тем не менее остается проблема управления подобными зависимостями, особенно если это касается больших приложений. Нередко для установки зависимостей в подобных системах используются специальные контейнеры - IoC-контейнеры (Inversion of Control). Такие контейнеры служат своего рода фабриками, которые устанавливают зависимости между абстракциями и конкретными объектами и, как правило, управляют созданием этих объектов.

Преимуществом ASP.NET Core в этом отношении является то, что фреймворк уже по умолчанию имеет встроенный контейнер внедрения зависимостей, который представлен интерфейсом **IServiceProvider**. А сами зависимости еще называются сервисами, собственно поэтому контейнер можно назвать **провайдером сервисов**. Этот контейнер отвечает за сопоставление зависимостей с конкретными типами и за внедрение зависимостей в различные объекты.

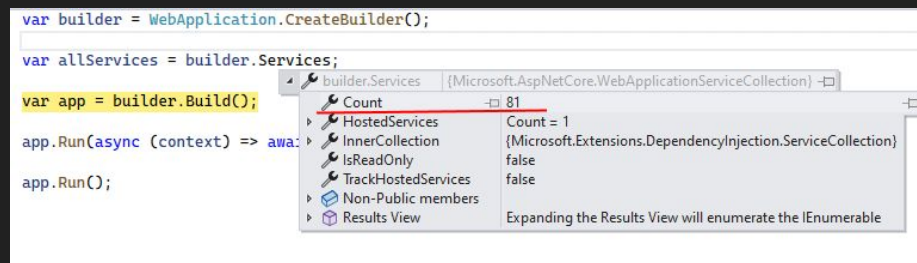
Установка встроенных сервисов фреймворка

За управление сервисами в приложении в классе **WebApplicationBuilder** определено свойство **Services**, которое представляет объект **IServiceCollection** - коллекцию сервисов:

```
WebApplicationBuilder builder =  
WebApplication.CreateBuilder();  
IServiceCollection allServices = builder.Services; //  
коллекция сервисов
```

И даже если мы не добавляем в эту коллекцию никаких сервисов, **IServiceCollection** уже содержит ряд сервисов по умолчанию

Как видно на скриншоте, в коллекции **IServiceCollection** 81 сервис, который мы можем использовать в приложении. Это такие сервисы, как **ILogger<T>**, **ILoggerFactory**, **IWebHostEnvironment** и ряд других. Они добавляются по умолчанию инфраструктурой ASP.NET Core. И мы их можем использовать в различных частях приложения.

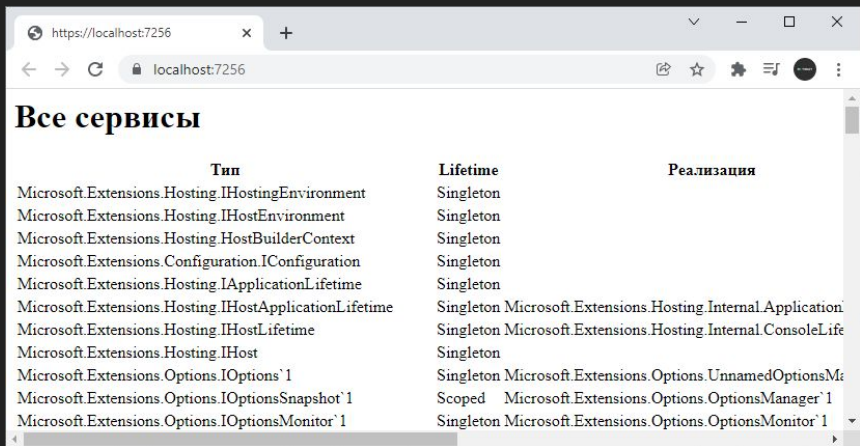


Информация о сервисах

Каждый сервис в коллекции `IServiceCollection` представляет объект **ServiceDescriptor**, который несет некоторую информацию. В частности, наиболее важные свойства этого объекта:

- **ServiceType**: тип сервиса
- **ImplementationType**: тип реализации сервиса
- **Lifetime**: жизненный цикл сервиса

Например, получим все сервисы, которые добавлены в приложение



Тип	Lifetime	Реализация
Microsoft.Extensions.Hosting.IHostingEnvironment	Singleton	
Microsoft.Extensions.Hosting.IHostEnvironment	Singleton	
Microsoft.Extensions.Hosting.HostBuilderContext	Singleton	
Microsoft.Extensions.Configuration.IConfiguration	Singleton	
Microsoft.Extensions.Hosting.IApplicationLifetime	Singleton	
Microsoft.Extensions.Hosting.IHostApplicationLifetime	Singleton	Microsoft.Extensions.Hosting.Internal.ApplicationLifetime
Microsoft.Extensions.Hosting.IHostLifetime	Singleton	Microsoft.Extensions.Hosting.Internal.ConsoleLifetime
Microsoft.Extensions.Hosting.IHost	Singleton	
Microsoft.Extensions.Options.IOptions`1	Singleton	Microsoft.Extensions.Options.UnnamedOptionsManager`1
Microsoft.Extensions.Options.IOptionsSnapshot`1	Scoped	Microsoft.Extensions.Options.OptionsManager`1
Microsoft.Extensions.Options.IOptionsMonitor`1	Singleton	Microsoft.Extensions.Options.OptionsMonitor`1

```
using System.Text;

var builder = WebApplication.CreateBuilder();

var services = builder.Services;

var app = builder.Build();

app.Run(async context =>
{
    var sb = new StringBuilder();
    sb.Append("<h1>Все сервисы</h1>");
    sb.Append("<table>");

    sb.Append("<tr><th>Тип</th><th>Lifetime</th><th>Реализация</th></tr>");
    foreach (var svc in services)
    {
        sb.Append("<tr>");
        sb.Append($"<td>{svc.ServiceType.FullName}</td>");
        sb.Append($"<td>{svc.Lifetime}</td>");

        sb.Append($"<td>{svc.ImplementationType?.FullName}</td>");
        sb.Append("</tr>");
    }
    sb.Append("</table>");
    context.Response.ContentType =
"text/html; charset=utf-8";
    await context.Response.WriteAsync(sb.ToString());
});

app.Run();
```

Регистрация встроенных сервисов ASP.NET Core

Кроме ряда подключаемых по умолчанию сервисов ASP.NET Core имеет еще ряд встроенных сервисов, которые мы можем подключать в приложение при необходимости. Все сервисы и компоненты middleware, которые предоставляются ASP.NET по умолчанию, регистрируются в приложение с помощью методов расширений IServiceCollection, имеющих общую форму Add[название_сервиса].

Например:

```
var builder = WebApplication.CreateBuilder();  
builder.Services.AddMvc();
```

Для объекта IServiceCollection определено ряд методов расширений, которые начинаются на *Add*, как, например, AddMvc(). Эти методы добавляют в объект IServiceCollection соответствующие сервисы. Например, AddMvc() добавляет в приложение сервисы MVC, благодаря чему мы сможем их использовать в приложении.

Создание сервисов

Фреймворк ASP.NET Core предоставляет ряд встроенных сервисов, которые мы можем использовать. Но также мы можем создавать свои собственные сервисы. Рассмотрим, как это сделать.

Определим новый интерфейс **ITimeService**, который предназначен для получения времени:

```
interface ITimeService
{
    string GetTime();
}
```

И также определим два класса, которые будут реализовать данный интерфейс. Первый класс будет называться **ShortTimeService** и будет возвращать текущее время в формате hh:mm(то есть часы и минуты):

```
// время в формате hh:mm
class ShortTimeService : ITimeService
{
    public string GetTime() => DateTime.Now.ToShortTimeString();
}
```

Второй класс будет называться **LongTimeService** - он будет возвращать время в формате hh:mm:ss:

```
// время в формате hh:mm:ss
class LongTimeService : ITimeService
{
    public string GetTime() => DateTime.Now.ToString();
}
```

Создание сервисов

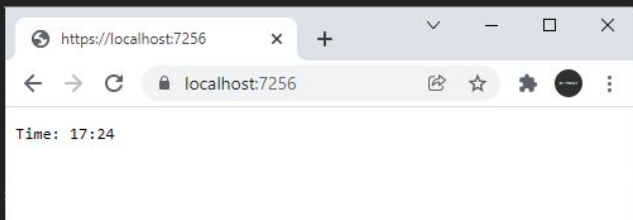
Теперь добавим в коллекцию сервисов сервис `ITimeService` и используем его в приложении

После добавления сервиса его можно получить и использовать в любой части приложения. Для получения сервиса могут применяться различные способы в зависимости от ситуации. В данном случае используется свойство `app.Services`, которое предоставляет провайдер сервисов - объект `IServiceProvider`. Для получения сервиса у провайдера сервиса вызывается метод `GetService()`, который типизируется типом сервиса:

```
var timeService =  
app.Services.GetService<ITimeService>();
```

После получения сервиса мы можем использовать его.

```
await context.Response.WriteAsync($"Time:  
{timeService?.GetTime()}");
```



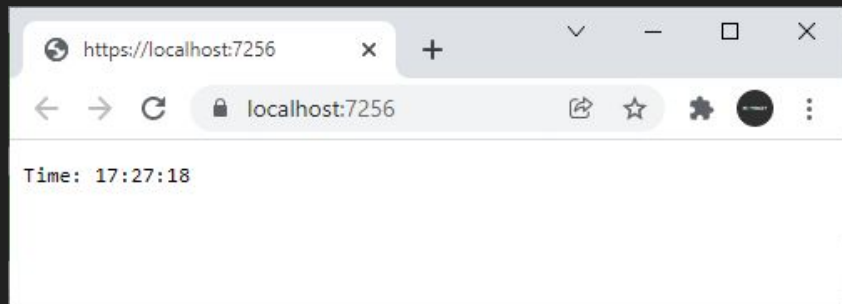
```
var builder = WebApplication.CreateBuilder();  
  
builder.Services.AddTransient<ITimeService, ShortTimeService>();  
  
var app = builder.Build();  
  
app.Run(async context =>  
{  
    var timeService = app.Services.GetService<ITimeService>();  
    await context.Response.WriteAsync($"Time:  
{timeService?.GetTime()}");  
});  
  
app.Run();  
  
interface ITimeService  
{  
    string GetTime();  
}  
  
// время в формате hh:mm  
class ShortTimeService : ITimeService  
{  
    public string GetTime() => DateTime.Now.ToShortTimeString();  
}  
  
// время в формате hh:mm:ss  
class LongTimeService : ITimeService  
{  
    public string GetTime() => DateTime.Now.ToLongTimeString();  
}
```

Создание сервисов

Поскольку метод `AddTransient` установил зависимость между `ITimeService` и `ShortTimeService`, то в браузере выводится текущее время в формате "hh:mm". Мы можем поменять тип, сопоставляемый с `ITimeService`:

```
builder.Services.AddTransient<ITimeService,  
LongTimeService>();
```

И в этом случае мы увидим другое сообщение.



Сервис как конкретный класс

При этом необязательно разделять определение сервиса в виде интерфейса и его реализацию. Сам термин "сервис" в данном случае может представлять любой объект, функциональность которого может использоваться в приложении.

Например, определим новый класс `TimeService`:

```
public class TimeService
{
    public string GetTime() =>
        DateTime.Now.ToShortTimeString();
}
```

Данный класс определяет один метод `GetTime()`, который возвращает текущее время.

Используем этот класс в качестве сервиса

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<TimeService>();

var app = builder.Build();
app.Run(async context =>
{
    var timeService = app.Services.GetService<TimeService>();
    await context.Response.WriteAsync($"Time: {timeService?.GetTime()}");
});

app.Run();

public class TimeService
{
    public string GetTime() => DateTime.Now.ToShortTimeString();
}
```

Расширения для добавления сервисов

Нередко для сервисов создают собственные методы добавления в виде методов расширения для интерфейса `IServiceCollection`. Например, создадим подобный метод для сервиса `TimeService`:

```
public static class ServiceProviderExtensions
{
    public static void AddTimeService(this
IServiceCollection services)
    {
        services.AddTransient<TimeService>();
    }
}
```

И теперь используем этот метод

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTimeService();

var app = builder.Build();
app.Run(async context =>
{
    var timeService = app.Services.GetService<TimeService>();
    context.Response.ContentType = "text/html; charset=utf-8";
    await context.Response.WriteAsync($"Текущее время:
{timeService?.GetTime()}");
});

app.Run();

public class TimeService
{
    public string GetTime() => DateTime.Now.ToShortTimeString();
}

public static class ServiceProviderExtensions
{
    public static void AddTimeService(this IServiceCollection services)
    {
        services.AddTransient<TimeService>();
    }
}
```